

CI/CD & Azure DevOps

1. What is CI/CD?

CI/CD stands for Continuous Integration and Continuous Delivery (or Continuous Deployment). It's a method of automating the process of building, testing, and releasing software so that code changes go from a developer's machine to production quickly and reliably.

Before CI/CD, developers would write code for weeks, merge everything at once, and hope it worked. That caused big messy conflicts and slow releases. CI/CD breaks that into small, frequent, automated steps.

Continuous Integration (CI)

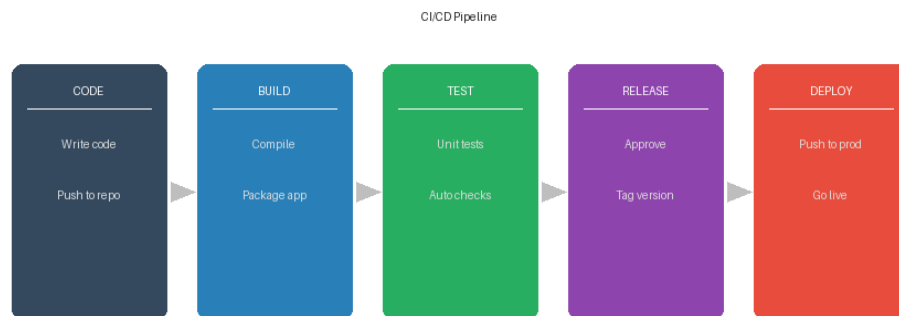
CI means developers merge their code into a shared branch frequently — sometimes multiple times a day. Every time code is pushed, an automated pipeline runs to build and test it immediately.

- Catches bugs early — right when the code is written, not weeks later.
- Everyone's code is always in sync — no massive merge conflicts.
- Automated tests run on every commit — no manual testing required.

Continuous Delivery (CD)

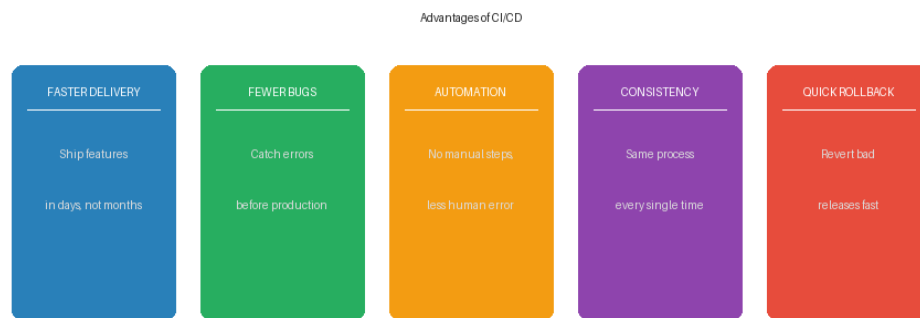
CD means once code passes all tests, it's automatically packaged and ready to be deployed to production at any time. In Continuous Deployment (one step further), it goes live automatically without any manual approval.

- The release process is automated — no manual steps to push code out.
- You can deploy multiple times a day if needed.
- Every build that passes tests is a valid release candidate.



2. Advantages of CI/CD Pipelines

- **Faster releases:** Features and bug fixes reach users in hours or days, not weeks.
- **Fewer bugs in production:** Automated tests catch problems before they ever reach users.
- **Less manual work:** Builds, tests, and deployments are all automated. Fewer human errors.
- **Consistent process:** Every release goes through the exact same steps — no shortcuts.
- **Easy rollback:** If something breaks in production, you can revert to the previous version quickly.
- **Better team confidence:** Developers can push code knowing it'll be tested automatically before it causes problems.
- **Shorter feedback loop:** You know within minutes if your code broke something, not days later.



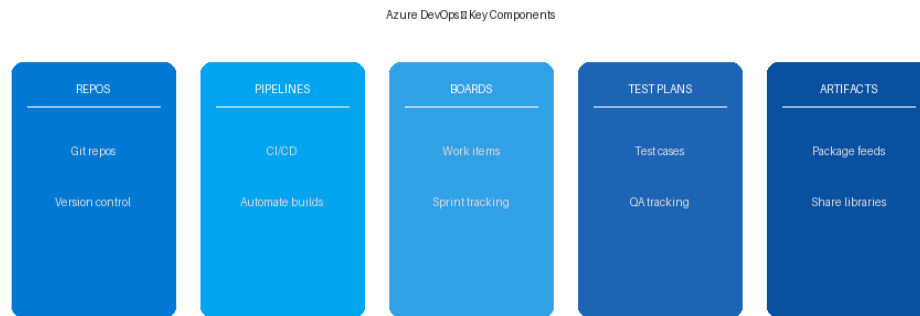
3. CI/CD from an Azure DevOps Perspective

Azure DevOps is Microsoft's platform for managing the entire software development lifecycle. It has everything you need to set up CI/CD pipelines and collaborate as a team — all in one place.

Key Components of Azure DevOps

- **Azure Repos:** Git-based version control. Developers push code here. Every push can trigger a pipeline.
- **Azure Pipelines:** The heart of CI/CD in Azure DevOps. Define your build and deploy steps using YAML files or a visual editor.
- **Azure Boards:** Track work items, bugs, and sprints. Connects with pipelines to link code commits to tasks.

- **Azure Test Plans:** Manage test cases and track test results across your pipeline runs.
- **Azure Artifacts:** Store and share packages (NuGet, npm, Maven) used across your pipelines.



How a Typical Azure DevOps Pipeline Works

A developer pushes code to Azure Repos. This triggers an Azure Pipeline automatically. The pipeline runs in stages:

- **Build stage:** The code is compiled and packaged into a deployable artifact.
- **Test stage:** Automated unit and integration tests run against the build.
- **Release stage:** If tests pass, the artifact is pushed to a staging or production environment.
- **Approvals:** You can add manual approval gates before production deployments.

YAML Pipelines

Azure Pipelines uses YAML files (azure-pipelines.yml) to define the pipeline as code. This means your pipeline configuration lives in the same repo as your application code — versioned, reviewable, and repeatable.

- Define triggers — which branches kick off a pipeline run.
- Set up stages — build, test, deploy — all in one file.
- Use variables and environments to manage different deployment targets (dev, staging, prod).

Azure DevOps Environments

Environments in Azure DevOps represent where your code gets deployed — for example, Development, QA, Staging, and Production. Each environment can have:

- Approval checks — a human must approve before deployment proceeds.

- Gates — automated checks like monitoring thresholds or test results.
- History — a full audit trail of every deployment to that environment.

Quick Reference

- **CI** — Automatically build and test code every time it's pushed.
- **CD** — Automatically deliver tested code to a deployable or live state.
- **Azure Pipelines** — Define CI/CD using YAML. Triggered by code pushes to Azure Repos.
- **Azure DevOps** — One platform: Repos + Pipelines + Boards + Test Plans + Artifacts.